ResilientSDN

Congestion- and attack-aware rerouting for Software-Defined Networks

---

Prepared for presentation by **Doni Adithya**

Status snapshot: 2026-10-01

1. Problem Statement

Software-Defined Networks centralize routing decisions in a controller, which makes them flexible but also exposes a weak point: an attacker can flood a link to force congestion, and a routing scheme that reacts carelessly can make things worse — oscillating between paths, moving legitimate traffic onto other already-busy links, or simply not reacting fast enough.

ResilientSDN is a Ryu-based SDN controller that detects congestion caused by DDoS-style and link-flooding attacks and reroutes only the traffic that needs to move, onto paths that can actually absorb it, without flapping back and forth.

What it is measured against

- **Static routing** — shortest path fixed at startup, never adapts.
- **Simple reactive controller** — reroutes on a single threshold crossing, no capacity check, no hysteresis.
- **QoSentry** — a learned-policy baseline, included as an interface (no trained model available yet).

Test conditions

Four topologies — a Fat-tree datacenter network, and three wide-area research/backbone networks (Internet2/OS3E, GÉANT, NSFNET) — under eleven attack scenarios (S1–S11): plain floods at increasing rates, distributed floods, a sustained overload, and four adversarial patterns designed to evade threshold-based detection (pulse, cooldown-timed, capacity presaturation, randomised thresholds, long overload).
The question being answered: *does scoring candidate paths on capacity, latency and rule-change cost, with hysteresis to prevent flapping, actually recover faster and more cleanly than a naive reactive scheme — and does it hold up against an attacker who knows how the detector works?*

2. Tech Stack — What and Why

Component	Role in this project	Why this one
Ryu (Python SDN framework)	Implements the controller logic: monitoring, detection, scoring, flow rule installation	Pure Python, single pip install, easy to read and modify (scoring, hysteresis) is testa
Mininet	Emulates the network — hosts, switches and links with separate network namespaces	Gives a real, isolated network on a laptop, no hardware needed, and every topology c
Open vSwitch	The actual switch software Mininet uses; executes the flow rules the controller installs	Production-grade OpenFlow switch, so the rules tested here are th
NetworkX	Graph representation of each topology; computes candidate paths and shortest paths	Path computations and topology utilities are already solved co
iperf3 / tcpreplay / hping3	Generate legitimate traffic, replay real-world traces, and generate attack floods	Needed for traffic that behaves like real TCP/UDP flows and real a
Pandas / SciPy	Turn thousands of per-run CSV rows into means, 95% confidence intervals, and significance tests (Welch's	The standard tools for exactly this kind of analysis
Matplotlib	Generates the result figures (throughput, loss, recovery time, CDF, ablation, heatmap)	Saves to PDF/PNG, labeling that looks professional, and reproduces identically on every r
Docker / WSL2	Two interchangeable ways to run Mininet + Open vSwitch + Ryu, since they need Linux kernel features	Windows can't run Mininet natively. Docker gives a clean, disposa

Python 3.9 runs the Ryu controller specifically (Ryu is unmaintained and breaks on newer eventlet versions);
Python 3.11 runs everything else — tests, analysis, plotting — since those have no such constraint.

3. Progress

Build status by phase

#	Phase	Status
1	Environment setup (Windows venv, Docker, WSL2)	Done
2	Network topologies: Fat-tree, Internet2, GEANT + any-topology support (NSFNET and others via Topology Zoo)	Done
3	Monitoring module (link/flow stats, latency probing)	Done
4	Congestion detector (N-consecutive trigger)	Done
5	Path evaluation engine (candidates, capacity filter, scoring)	Done
6	Flow rule manager	Done
7	Main controller (live reroute, e.g. recovers in ~8s on Fat-tree)	Done
8	Baselines: static, simple reactive; QoSentry interface only	Done (QoSentry needs a trained policy)
9	Attack simulation, scenarios S1-S11	Done
10	Evaluation automation: full campaign + ablation A-D	Done -- 610 runs, 0 failed
11	Results and visualisation: figures, LaTeX tables, PDF reports	Done

Headline results

- **Fat-tree, high single-flow attack (S3):** ResilientSDN recovers in about 8 seconds; static routing never recovers; simple reactive causes 27 oscillations and 7 secondary congestion events while doing it.
- **WAN capacity-presaturation attack (S9):** simple reactive loses 12.6-18.3% of legitimate traffic (200-270 reroutes); ResilientSDN loses 0% (3-5 reroutes) — the capacity filter correctly refuses to move traffic onto links the attacker has pre-filled.
- **Pulse attack (S7), designed to evade the detector:** all three systems show 0 loss; ResilientSDN correctly does nothing, while simple reactive still makes 48 pointless reroutes.
- **127 unit tests passing;** full release check (lint, image build, connectivity, security review, dependency audit) passed; two Medium security findings (forged latency probes, packet-in flood) found and fixed.

Known limitation, investigated and documented

On Internet2 (WAN), one attack scenario (S11) keeps the target link above the congestion threshold: the scoring function sums utilisation over every hop, and Internet2's detour paths are 7-13 hops long, so every detour scores worse than leaving the attack flow in place. Two fix attempts were tried and rejected because they traded this problem for worse loss elsewhere on a different link. The decision taken: keep the current behaviour (it protects legitimate traffic, which still sees only 0.6% loss vs. 9.4% for static routing) and report the limitation honestly rather than hide it.

What's left

- Decide whether to pursue a different fix for the Internet2 limitation, or keep it as documented (current recommendation: keep).
- Open items from the original plan: an attack-flow classifier, an ECMP-based static baseline, a trained QoSentry policy, and real CAIDA/MAWI trace replay.